

روش‌های دور زدن انواع کپچا و راهکارهای امنیتی مقابله با آن

مقدمه

کپچا (CAPTCHA) مکانیزمی امنیتی است که برای تمایز بین انسان و ربات در خدمات وب به کار می‌رود ¹. این آزمون‌ها معمولاً شامل چالش‌هایی هستند که حل آن‌ها برای انسان ساده ولی برای اسکریپت‌ها یا بات‌های خودکار دشوار است (مانند تشخیص حروف تحریف‌شده، شناسایی تصاویر خاص یا حل معماهای ساده). هدف کپچا جلوگیری از فعالیت‌های مخرب خودکار مانند ارسال هرزنامه، حملات جستجوی فراگیر رمز عبور، ساخت حساب‌های جعلی و سایر سوءاستفاده‌های رباتیک است ². با وجود نقش مهم کپچا در امنیت وب، مهاجمان همواره به دنبال روش‌هایی برای دور زدن کپچا بوده‌اند و پیشرفت هوش مصنوعی و ترفندهای برنامه‌نویسی، برخی از این مکانیزم‌ها را آسیب‌پذیر کرده است ³. در این مستند جامع، ابتدا تمامی روش‌های شناخته‌شده بایپس و دور زدن انواع کپچا را بررسی می‌کنیم. سپس راه‌حل‌های امن‌سازی کپچاها و جلوگیری از دور زدن آن‌ها را مرور کرده و در نهایت اصول طراحی کپچایی را بحث می‌کنیم که از همه جهات مقاوم بوده و غیرقابل دور زدن باشد.

روش‌ها و تکنیک‌های دور زدن کپچاها

با گذشت زمان، مهاجمان روش‌های متنوعی برای شکست دادن یا دور زدن کپچا ابداع کرده‌اند. این روش‌ها را می‌توان در چند دسته کلی جای داد: سوءاستفاده از ضعف‌های پیاده‌سازی کپچا، حل خودکار کپچا با استفاده از هوش مصنوعی یا خدمات جانبی، و دور زدن غیرمستقیم کپچا از طریق تقلید رفتارهای کاربر واقعی برای جلوگیری از نمایش آن. در ادامه هر یک از این رویکردها با جزئیات بررسی می‌شود:

۱. سوءاستفاده از ضعف‌های پیاده‌سازی (مشکلات منطقی و طراحی)

بسیاری از کپچاها سفارشی یا پیاده‌سازی‌شده به صورت نادرست، دارای ضعف‌های منطقی هستند که امکان دور زدن آن‌ها را بدون نیاز به حل واقعی چالش فراهم می‌کند. مهاجمان حرفه‌ای (به‌ویژه در تست نفوذ وب) همواره این ضعف‌ها را جستجو می‌کنند. **نمونه‌هایی از این ضعف‌های رایج عبارت‌اند از:**

- **استفاده‌ی مجدد از پاسخ کپچای قبلی:** در برخی سیستم‌ها، کد کپچا پس از یک بار حل شدن، برای درخواست‌های بعدی نیز معتبر باقی می‌ماند. در نتیجه مهاجم می‌تواند پاسخ حل‌شده‌ی یک کپچا (مثلاً "ABC123") را بارها ارسال کرده و هر بار بدون نیاز به حل کپچا تأیید بگیرد ⁴ ⁵. این وضعیت زمانی رخ می‌دهد که سرور برای هر جلسه یا هر درخواست، کپچای جدیدی تولید نکند یا **تازه بودن** پاسخ کپچا را صحت‌سنجی نکند.
- **پذیرش ارسال کپچا به صورت تهی یا غیرفعال:** دیده شده برخی فرم‌ها چنانچه مقدار کپچا ارسال نشود، باز هم درخواست را می‌پذیرند ⁶. این یک باگ منطقی است؛ به عنوان مثال درخواست HTTP حاوی `captcha=` (بدون مقدار) ممکن است در غیاب اعتبارسنجی صحیح، توسط سرور پذیرفته شود.
- **ارسال داده از مسیر یا فرمت متفاوت:** اگر برنامه‌ی وب تنها در یک مسیر یا نوع درخواست کپچا را اعتبارسنجی می‌کند، مهاجم می‌تواند از مسیر جایگزین استفاده کند. برای نمونه، برخی سیستم‌ها کپچا را فقط

در فرم ارسال HTML بررسی می‌کنند؛ در حالی که ارسال همان داده از طریق یک API JSON ممکن است اعتبارسنجی کپچا را دور بزند ⁷ . تغییر روش درخواست از POST به GET یا سایر متدها نیز گاهی باعث عبور از منطق اعتبارسنجی می‌شود ⁸ .

دستکاری سربرگ‌ها و پارامترها: افزودن یا تغییر سربرگ‌های HTTP (مانند X-Forwarded-For یا X-Remote-IP) ممکن است سرور را درباره منبع درخواست گمراه کرده و مکانیزم کپچا را دور بزند ⁹ . همچنین در مواردی، تغییر مقادیر سایر پارامترهای فرم (مثل شناسه کاربر) می‌تواند منجر به نقص در منطق اعتبارسنجی کپچا شود ¹⁰ . این حالت‌ها معمولاً ناشی از طراحی نادرست گردش کار کپچا و عدم در نظرگیری تمامی حالت‌های ممکن توسط برنامه‌نویسان است.

اشکالات منطق کدنویسی در سمت سرور: برای مثال، در یک سناریوی افشاشده، وبسایتی از reCAPTCHA گوگل استفاده می‌کرد اما برنامه‌نویس به‌جای بررسی محتوای پاسخ JSON گوگل، فقط کد وضعیت HTTP را چک می‌کرد ¹¹ ¹² . از آنجا که سرویس reCAPTCHA در هر حالت کد OK ۲۰۰ برمی‌گرداند، مهاجم می‌توانست هر مقداری (حتی رشته‌ی "InvalidAnswerOfCAPTCHA") را به سرور ارسال کند و پاسخ ۲۰۰ را به‌عنوان قبولی تفسیر کرده و کپچا را عملاً بی‌اثر کند ¹² . در مثالی دیگر، خطای برنامه‌نویسی در استفاده از بلوک‌های شرطی if/else باعث می‌شد حتی پاسخ نادرست کپچا نیز به‌اشتباه موفق تلقی شود ¹³ . همچنین مواردی گزارش شده که کپچا در سمت کاربر نمایش داده شده ولی نتیجه آن اصلاً در سمت سرور بررسی نمی‌شد ¹⁴ - کافایت مهاجم هر مقداری وارد کرده و ارسال کند تا بدون اعتبارسنجی عبور کند. این قبیل **پیاده‌سازی‌های اشتباه** از لحاظ امنیتی به‌شدت خطرناک‌اند و عملاً وجود کپچا را بی‌اثر می‌کنند.

طراحی‌های ضعیف کپچا: حتی اگر پیاده‌سازی فنی بدون باگ باشد، **نواقص در طراحی مکانیزم کپچا** می‌تواند امکان دور زدن را فراهم کند. برای مثال، کپچایی را در نظر بگیرید که یک مسأله ریاضی ساده (جمع یا تفریق اعداد) را در خود صفحه نمایش می‌دهد. اگر پرسش ریاضی مستقیماً در کد HTML صفحه وجود داشته باشد، ربات می‌تواند به‌راحتی آن را از سورس صفحه استخراج و محاسبه کرده و پاسخ را ارسال کند ¹⁵ ¹⁶ . نمونه‌ی دیگر، کپچایی با بانک محدودی از سوالات است؛ فرضاً تنها ۱۰ سؤال ممکن دارد و به صورت چرخشی از میان آن‌ها انتخاب می‌شود ¹⁷ . در این حالت مهاجم می‌تواند با تلاش دستی یا خودکار، پاسخ تمام آن ۱۰ سؤال را یک بار استخراج و یاد بگیرد و سپس هر بار با تطبیق تصویر سؤال، پاسخ صحیح از قبل ذخیره‌شده را ارسال کند ¹⁸ . چنین کپچاهایی به دلیل محدودیت طراحی، به‌سادگی **مهندسی معکوس** یا حفظ می‌شوند و خاصیت چالش‌برانگیز بودن خود را از دست می‌دهند.

با توجه به موارد فوق، مشخص می‌شود که هرگونه **اشتباه در طراحی یا پیاده‌سازی کپچا** می‌تواند به دور زدن آن منجر شود. گزارش‌های متعددی در فضای امنیتی وجود دارد که جزئیات کشف این نوع آسیب‌پذیری‌ها را شرح می‌دهد ¹⁹ ²⁰ . برای مثال یک محقق امنیتی در یک وب‌اپ بانکی کشف کرد که شناسه‌ی یکتای کپچا (CaptchaUniqueId) پس از یک بار استفاده، منقضی نمی‌شود و با ارسال مجدد همان شناسه به همراه پاسخ قبلی، می‌توان بدون حل کپچا به مرحله بعد رفت ²¹ ²² . این **حملات بازپخش** به خاطر عدم اعتبارسنجی نشست/زمان رخ می‌دهند و اهمیت مدیریت جلسه در کپچا را نشان می‌دهند.

۲. حل خودکار کپچا با ابزارهای هوش مصنوعی و OCR

دسته‌ی دوم روش‌های باییس کپچا، مستقیماً **حل کردن خودکار چالش کپچا** با بهره‌گیری از فناوری‌های پیشرفته است. در واقع در این روش‌ها، به‌جای سوءاستفاده از باگ، تلاش می‌شود خود کپچا مانند یک انسان حل شود - چه توسط نرم‌افزارهای هوشمند و چه با دور زدن کار به انسان‌های دیگر. طی سال‌های اخیر پیشرفت چشمگیری در این

زمینه رخ داده است و بسیاری از کپچاهای سنتی دیگر **سد محکم و غیرقابل نفوذی برای ماشین‌ها محسوب نمی‌شوند** ²³.

تشخیص کاراکترهای متن کپچا با OCR: کپچاهای متنی که کاربر را ملزم به خواندن حروف/اعداد کج و معوج و وارد کردن آن‌ها می‌کنند از قدیمی‌ترین انواع هستند. امروزه سامانه‌های OCR (تشخیص نوری حروف) و بینایی ماشین بسیار پیشرفته شده‌اند و می‌توانند متن‌های تحریف‌شده را با دقت قابل توجهی بازشناسی کنند ²³. پژوهش‌های امنیتی نشان می‌دهد نرخ موفقیت ابزارهای OCR و مدل‌های هوش مصنوعی در خواندن متن کپچاهای پیچیده بین ۸۰٪ تا ۹۵٪ است ²⁴. به عنوان مثال، پروژه‌های متن‌بازی پیاده‌سازی شده که با استفاده از شبکه‌های عصبی کانولوشن، کپچاهای متنی را به‌طور خودکار حل می‌کنند ²⁵ ²⁶. هرچند طراحان کپچا با افزودن اعوجاج شدید، نویز، پس‌زمینه‌ی مختل‌کننده و تکنیک‌های پیچیده‌ی دیگر سعی در کاهش دقت OCR دارند، اما این یک مسابقه‌ی تسلیحاتی است که با قوی‌تر شدن الگوریتم‌های یادگیری ماشین، توانایی شکست کپچاهای متنی نیز افزایش می‌یابد ²³.

تشخیص اشیاء در کپچاهای تصویری: کپچاهای رایج امروزی - نظیر reCAPTCHA v2 گوگل یا hCaptcha - اغلب کاربر را وادار به انتخاب تصاویر خاص (مانند همه‌ی عکس‌های حاوی چراغ راهنمایی، عابر پیاده، اتوبوس و غیره) می‌کنند. در نگاه اول این یک چالش بینایی برای هوش مصنوعی است؛ اما مدل‌های پیشرفته‌ی تشخیص تصویر (خصوصاً شبکه‌های عصبی عمیق) اکنون در بسیاری موارد از دقت انسان پیشی گرفته‌اند ²⁷ ²⁸. **داده‌های آموزشی کافی**، یک مدل یادگیری عمیق می‌تواند اجسام مورد نظر را در تصاویر کپچا شناسایی کند. تخمین زده می‌شود که نرخ موفقیت مدل‌های بینایی ماشین در حل کپچاهای تصویری پیچیده حدود ۷۰٪ تا ۹۰٪ باشد ²⁹. ابزارها و کتابخانه‌های متن‌باز نیز برای این منظور توسعه یافته‌اند. برای مثال، پروژه‌ای به نام Buster به صورت افزونه مرورگر موجود است که چالش‌های صوتی reCAPTCHA را با تبدیل صوت به متن حل می‌کند ³⁰ (در بخش کپچا صوتی توضیح داده خواهد شد)؛ یا پروژه‌های دیگری با OpenCV تلاش کرده‌اند کپچاهای تصویری **پازلی/اسلایدی** را بشکنند. یکی از روش‌های محبوب کپچا در سایت‌های آسیای شرقی، پازل‌های تصویری جابجایی قطعات است (مانند کپچای معروف Geetest)؛ در این نوع، کاربر باید قطعه‌ای از تصویر را در محل مناسب قرار دهد. محققان موفق شده‌اند با تکنیک‌های بینایی کامپیوتر (نظیر **تشخیص لبه و تطبیق الگو**)، محل دقیق قطعه در تصویر پس‌زمینه را پیدا کرده و کپچاهای اسلایدی را خودکار حل کنند ³¹. بنابراین، کپچاهای تصویری نیز از تیغ هوش مصنوعی در امان نبوده‌اند.

حل کپچاهای صوتی با پردازش گفتار: برای دسترسی‌پذیری کاربران نابینا یا دارای مشکل بینایی، بسیاری از سیستم‌های کپچا گزینه‌ی **شنیداری** ارائه می‌دهند که شامل خواندن حروف یا اعداد توسط رایانه با صدای مخدوش است. جالب آن‌که این مسیر صوتی اغلب راهی ساده‌تر برای ربات‌ها فراهم می‌کند. تکنیک رایج مهاجمان برای کپچاهای صوتی (از جمله reCAPTCHA Audio Challenge) آن است که فایل صوتی کپچا را دریافت کرده و توسط **الگوریتم‌های تشخیص گفتار** یا حتی سپردن آن به API‌هایی مثل Google Speech-to-Text، متن آن را استخراج کنند ³². پروژه‌ی معروفی به نام *Uncaptcha* نشان داد که می‌توان کپچای صوتی گوگل را با حدود ۸۵٪ دقت فقط در چند ثانیه به متن تبدیل کرد ³⁰. به طور کلی، پردازش صوت و گفتار اکنون به حدی پیشرفته است که تشخیص اعداد خوانده‌شده در اکثر کپچاهای صوتی با دقت بالای ۹۰٪ ممکن است ²⁹. این امر چالشی جدی برای کپچاهای صوتی محسوب می‌شود.

بهره‌گیری از ربات‌های شبیه‌ساز مرورگر: برخی راهکارهای دور زدن، به جای پرداختن مستقیم به حل معمای کپچا، رویکرد **شبیه‌سازی کاربر واقعی** را در پیش می‌گیرند. برای نمونه، ابزارهای اتوماسیون مرورگر (مانند Selenium، Puppeteer و Playwright) می‌توانند یک مرورگر واقعی را در حالت مخفی (Headless) اجرا کرده و با صفحه وب تعامل انسان‌گونه داشته باشند ³³. این ابزارها حتی قادرند حرکات ماوس، فشارهای صفحه‌کلید و تأخیرهای تصادفی را شبیه‌سازی کنند ³⁴. تا رفتاری طبیعی نمایش دهند. اگر کپچا صرفاً بر اساس رفتار کاربر تصمیم بگیرد (مثلاً reCAPTCHA v3 یک امتیاز اعتماد به رفتار می‌دهد)، یک اسکریپت هوشمند که کاملاً شبیه انسان عمل کند می‌تواند از نمایش کپچا جلوگیری کرده یا آن را به‌سادگی پشت‌سر بگذارد ³⁵ ³⁶. هرچه شبیه‌سازی رفتار دقیق‌تر باشد (استفاده از مرورگر واقعی، اجرای جاوااسکریپت، داشتن اثرانگشت مرورگر

شبیه انسان و ...)، احتمال تشخیص داده نشدن توسط سیستم ضربات بیشتر است ³⁷ ³⁸ . به این ترتیب، بات‌های پیشرفته می‌توانند بدون حل کردن مستقیم معمای کپچا، سازوکار آن را دور بزنند.

استفاده از پایگاه‌های داده و یادگیری قبلی: گاهی حل خودکار نیاز به هوش مصنوعی پیچیده ندارد؛ کافیه ربات از **دانش قبلی یا حافظه** استفاده کند. برای مثال، اگر کپچا یک سؤال چالشی از یک **مجموعه محدود** باشد (همان‌گونه که در مثال طراحی ضعیف #۲ اشاره شد)، ربات می‌تواند پاسخ‌های آن مجموعه را از قبل داشته باشد ³⁹ . یا اگر کپچا یک سؤال ساده متنی (مثلاً یک سؤال دانش عمومی یا معمای منطقی) بپرسد، ممکن است بتوان با یک پایگاه داده یا حتی با استفاده از *chatbot* های زبان (نظیر ChatGPT) به آن پاسخ داد ⁴⁰ . البته استفاده از مدل‌های زبانی هوشمند برای حل کپچا خود مبحث جالبی است؛ تحقیقات اخیر نشان داده حتی می‌توان **خود ChatGPT را فریب داد** تا نقش یک انسان حل‌کننده کپچا را ایفا کند و پاسخ کپچاها را در تعاملات خودکار ارائه دهد ⁴⁰ (هرچند توسعه‌دهندگان این مدل‌ها سعی در محدودسازی چنین سوءاستفاده‌هایی دارند). در مجموع، ربات‌ها هر راهی که بتوانند مثل یک انسان فکر کنند یا از دانش بشری استفاده کنند را برای عبور از کپچا به کار می‌گیرند.

۳. استفاده از نیروی انسانی و سرویس‌های حل کپچا

شاید عجیب به نظر برسد، اما یکی از **مؤثرترین و آسان‌ترین** روش‌های دور زدن کپچا، سپردن حل آن به خود انسان است! در این روش، ربات یا اسکریپت مخرب، کپچایی را که سر راهش قرار گرفته به یک **سرویس آنلاین حل کپچا** یا شبکه‌ای از انسان‌ها ارسال می‌کند و پاسخ آن را دریافت می‌کند. این سرویس‌ها معمولاً با اتکا بر نیروی انسانی ارزان‌قیمت (مثلاً کارگران آنلاین در کشورهای مختلف) حجم عظیمی از کپچاها را در زمان کوتاه حل می‌کنند ⁴¹ . برخی از معروف‌ترین سرویس‌های حل کپچا عبارت‌اند از: **2Captcha, Anti-Captcha, DeathByCaptcha** و ... که از طریق API در دسترس هستند ⁴² ⁴³ . هزینه‌ی حل هر کپچا در این سیستم‌ها بسیار ناچیز است (مثلاً حدود 1\$ به ازای ۱۰۰۰ کپچا، یعنی ۰٫۰۰۱ سنت برای هر کپچا) ⁴² . سرعت پاسخگویی نیز معمولاً بین ۵ تا ۲۰ ثانیه است ⁴⁴ . با توجه به دقت نزدیک به ۹۰٪ تا ۹۸٪ نیروی انسانی در حل کپچا ⁴⁴ ، عملاً این روش یک دور زدن تضمینی به‌شمار می‌آید؛ زیرا **هیچ الگوریتم هوش مصنوعی فعلی نمی‌تواند با دقت ۱۰۰٪ همه کپچاها را حل کند، اما انسان‌ها تقریباً همیشه موفق‌اند**. از دید یک مهاجم، ترکیب یک ربات خودکار با یک سرویس حل کپچا به این شکل است که هرگاه ربات با چالش کپچا مواجه شد، تصویر یا صوت کپچا را برای سرویس می‌فرستد؛ انسان سرویس‌دهنده آن را حل کرده و جواب (مثلاً متن شناسایی‌شده یا گزینه صحیح) را برمی‌گرداند؛ ربات نیز پاسخ را در وبسایت هدف وارد کرده و عبور می‌کند.

این روش شاید از لحاظ فنی **دور زدن کپچا به شیوه‌ای کاملاً مشروع** به نظر نیاید (چون در واقع کپچا واقعاً توسط یک انسان حل شده)، اما از منظر امنیتی همان اثر را دارد که ربات به هدف خود می‌رسد. به این پدیده گاهی شکستن کپچا از طریق نیروی کار انسانی یا **حملات مزارع کپچا** گفته می‌شود ⁴¹ . وجود این سرویس‌ها یک **حفره بنیادین** در مکانیزم کپچا ایجاد کرده است؛ مادامی که حل کردن کپچا برای انسان (هرچند با صرف اندکی زمان یا هزینه) ممکن است، مهاجم می‌تواند آن را برون‌سپاری کند. این سرویس‌ها با اتوماسیون کامل (ارائه API برای دریافت تصویر و ارسال پاسخ) کار را برای نفوذگران بسیار ساده کرده‌اند ⁴⁵ ⁴⁶ . به بیان روشن‌تر، حتی پیچیده‌ترین کپچاهای امروز (شامل reCAPTCHA نامرئی گوگل یا کپچاهای مبتنی بر رفتار) نیز اگر مستقیماً توسط یک انسان پاسخ داده شوند، شکست خواهند خورد. لذا امنیت کپچا **هرگز فراتر از توانایی یک انسان در حل آن نخواهد رفت**.

۳. جلوگیری از نمایش کپچا (دور زدن از طریق مخفی ماندن)

دسته‌ی چهارم رویکردها، در واقع به‌جای حمله به خود کپچا، بر **دیده نشدن کپچا برای ربات** متمرکز است. بسیاری از سرویس‌های مدرن کپچا (خصوصاً سرویس‌های ضدبات) به این صورت عمل می‌کنند که ابتدا رفتار و ویژگی‌های کاربر را می‌سنجند و فقط در صورتی که احتمال ربات بودن زیاد باشد، کپچا را نمایش می‌دهند ⁴⁷ . بنابراین اگر یک ربات بتواند خود را کاملاً شبیه یک کاربر عادی جا بزند و امتیاز اعتماد (trust score) بالایی کسب کند، ممکن است **هرگز با چالش کپچا مواجه نشود** ⁴⁸ . این روش بایس را می‌توان دور زدن صامت نامید، چرا که ربات نیازی به حل یا حمله‌ی مستقیم به کپچا ندارد و اساساً وجود کپچا را دور می‌زند.

برای پیاده‌سازی این ترفند، ربات‌نویس‌ها از تمام روش‌های ضردیابی و شبیه‌سازی استفاده می‌کنند:

- **پروکسی‌ها و IP‌های چرخشی:** یکی از فاکتورهای مشکوک‌کننده، ارسال درخواست‌های زیاد از یک IP است. مهاجمان با استفاده از شبکه‌ای از پروکسی‌های چرخشی، IP خود را مداوم تغییر می‌دهند تا از دید سرور هر درخواست از دستگاه کاربری متفاوت بیاید⁴⁹ ⁵⁰. این کار همچنین امکان دور زدن محدودیت نرخ (Rate Limiting) را می‌دهد. استفاده از پروکسی‌های رزیدنشال (وابسته به ISP‌های واقعی) یا IP‌های کشورهای متنوع، شانس تشخیص ربات را کمتر می‌کند⁴⁹.

- **دروغ‌پردازی درباره‌ی ویژگی‌های مرورگر:** ربات‌ها تلاش می‌کنند اثر انگشت مرورگر خود را مخفی یا انسانی کنند. این شامل تنظیم یا چرخش User-Agent (معرف نوع مرورگر و سیستم‌عامل)⁵¹، تنظیم Accept-Language مطابق منطقه، فعال کردن جاوااسکریپت، استفاده از هدرهای مشابه مرورگرهای واقعی و حتی جعل ردپاهای پیچیده‌تری مانند Canvas fingerprint، WebGL، یا تیتون، TLS fingerprint متفاوتی نسبت به مرورگر Chrome یک درخواست ساده با کتابخانه‌ی requests دارد و می‌تواند توسط سرویس‌های ضدبات علامت‌گذاری شود⁵². اما استفاده از ابزارهایی نظیر curl با TLS impersonation یا راه‌اندازی مرورگر واقعی در حالت خودکار، باعث می‌شود TLS fingerprint و هدرهای HTTP شما عملاً مشابه کاربران واقعی باشد⁵³ ⁵⁴. همچنین کاهش سرعت ارسال درخواست‌ها و پرهیز از رگبار زدن به سرور (تنظیم تأخیرهای تصادفی بین درخواست‌ها) از روش‌های ساده‌ای است که ربات را طبیعی‌تر جلوه می‌دهد⁵⁵.

- **دوری از تله‌ها و honeypot‌ها:** برخی وب‌سایت‌ها، علاوه بر کپچا، از فیلدهای مخفی یا لینک‌های تقلبی استفاده می‌کنند تا ربات‌های عجول را به دام بیندازند. این تله‌ها (honeypot) معمولاً برای کاربر انسان قابل مشاهده یا تأثیر نیستند، اما اگر ربات بدون دقت همه فیلدها را پر کند یا روی همه لینک‌ها کلیک کند، خود را لو می‌دهد⁵⁶. یک ربات هوشمند با شناسایی و نادیده گرفتن این honeypot‌ها می‌تواند از فعال شدن مکانیزم‌های ضدربات (از جمله کپچا) پیشگیری کند.

- **استفاده از API‌های رسمی و مسیرهای جایگزین:** گاهی بهترین راه دور زدن کپچا این است که اصلاً به صفحه‌ی وب حاوی کپچا نرویم! بسیاری از وب‌سایت‌ها API‌هایی برای عملکردهای خود دارند که نیازی به کپچا ندارند. برای مثال به‌جای اسکریپت کردن فرم یک سایت (که کپچا دارد) می‌توان از API رسمی آن سایت استفاده کرد⁵⁷. این روش اخلاقی‌ترین و تمیزترین روش نیز به حساب می‌آید (چرا که عملاً درخواست مشروعی به API زده می‌شود)، ولی همیشه در دسترس نیست. با این حال، یک متخصص اسکریپتینگ داده پیش از تلاش برای شکست کپچا، بررسی می‌کند که آیا داده یا عملیات مورد نظر از طریق یک API محافظت‌شده (با کلید API و نرخ محدودیت) قابل انجام است یا خیر، چون در این صورت اصل مشکل کپچا منتفی می‌شود⁵⁸.

به طور خلاصه، **بات‌های مدرن سعی می‌کنند اصلاً کار به مواجهه مستقیم با کپچا نکند**. آن‌ها با تقلید تمام جزئیات رفتار کاربران واقعی (از سطح شبکه تا سطح تعامل کاربری) احتمال نمایش کپچا را به حداقل می‌رسانند³⁶. البته سرویس‌های ضدبات نیز بیکار ننشسته و پیوسته الگوریتم‌های شناسایی خود را بهبود می‌دهند تا حتی این ربات‌های ماهر را نیز تشخیص دهند. با این حال، پنهان ماندن از دید کپچا هنری است که همواره مهاجمان در آن موفقیت‌هایی کسب می‌کنند و نباید دست‌کم گرفته شود.

روش‌های امن‌سازی کپچا و جلوگیری از بایپس شدن آن

با توجه به راه‌های متعدد برای شکست کپچا که در بخش قبل مطرح شد، اکنون سؤال اساسی برای توسعه‌دهندگان و مدافعان امنیت این است که چگونه می‌توان **کپچاهای امن‌تر و مقاوم‌تری ساخت**. در این بخش، راهکارهای عملی و **بهترین اقدامات (Best Practices)** برای امن‌سازی کپچاها را بررسی می‌کنیم. این توصیه‌ها به دو بخش کلی تقسیم می‌شوند: یکی **اصول طراحی کپچا** که مانع شکست آسان توسط ماشین یا انسان شود، و دیگری **اصول پیاده‌سازی**

صحیح که جلوی سوءاستفاده از باگ‌ها و منطق نادرست را بگیرد. همچنین تدابیر تکمیلی دیگری را نیز مرور می‌کنیم که امنیت کپچا را در برابر حملات شناخته‌شده بالا می‌برد.

۱. طراحی یک کپچا با امنیت ذاتی بالا

- **از مکانیزم‌های کپچای معتبر و آزموده‌شده استفاده کنید:** تجربه نشان داده است کپچاهای خانگی ساده (مانند سؤال ریاضی یا پرسش از یک مجموعه محدود) عموماً آسیب‌پذیر هستند ¹⁵ ¹⁷. توصیه می‌شود به جای اختراع دوباره‌ی چرخ، از سرویس‌ها و کتابخانه‌های کپچای معتبر بهره بگیرید ⁵⁹. برای مثال، **Google reCAPTCHA** (نسخه ۲ یا ۳) یا **hCaptcha** از گزینه‌های مطرح هستند که طراحی پیچیده‌تر و پشته‌های تحقیقاتی قوی‌تری دارند. این سرویس‌ها علاوه بر چالش‌های معمول، از تحلیل رفتار کاربر و یادگیری ماشین برای تشخیص بات استفاده می‌کنند و به مراتب **سخت‌تر قابل دور زدن** هستند ⁶⁰. هرچند حتی این‌ها هم شکست‌ناپذیر نیستند، ولی قطعاً امن‌تر از کپچای ساده چند تصویر ثابت یا متن تحریف‌شده‌ی تک‌مرحله‌ای عمل می‌کنند. اگر پروژه‌ی شما حساسیت بالایی دارد، می‌توانید سراغ نسخه‌های پیشرفته‌تر نیز بروید (مثلاً **reCAPTCHA Enterprise** یا **hCaptcha Enterprise**) که سازوکارهای ضوابط قوی‌تری دارند ⁶⁰.
- **از طراحی‌های تک‌بعدی و ایستا پرهیز کنید:** کپچایی ایمن‌تر است که **غیرقابل پیش‌بینی و چندلایه** باشد. مثلاً کپچایی که فقط یک متن ثابت با اعوجاج نشان می‌دهد، تک‌بعدی است؛ یک ربات می‌تواند با OCR صرفاً آن را بخواند. اما اگر همان کپچا علاوه بر متن، نیاز به پاسخ به یک سؤال ساده‌ی تصویری یا انتخاب گزینه‌ی صحیح داشته باشد، شکست دادنش سخت‌تر می‌شود. برخی سرویس‌ها کپچاهای ترکیبی ارائه می‌کنند (مثلاً ابتدا کاربر باید یک چک‌باکس "من ربات نیستم" را تیک بزند تا یک ارزیابی اولیه رفتار انجام شود، سپس در صورت مشکوک بودن، یک چالش تصویری یا متنی نمایش داده می‌شود). این لایه‌بندی باعث می‌شود ربات نتواند با یک راهکار ساده همه چیز را حل کند. همچنین **پویاسازی محتوا** مهم است: کپچا نباید مجموعه‌ی محدودی از پرسش‌ها یا تصاویر داشته باشد. تولید تصادفی چالش‌ها (چه متن و چه تصویر) و داشتن بانک بزرگی از موارد ممکن، کار حمله با روش‌های یادگیری قبلی را سخت می‌کند ⁶¹.
- **در نظر گرفتن دسترسی‌پذیری (Accessibility):** یک نکته مهم این است که کپچاهایی که صرفاً بر پیچیده‌سازی برای ماشین تمرکز می‌کنند، گاهی برای کاربران واقعی (به‌ویژه افراد دارای معلولیت) نیز دشوار می‌شوند ⁶² ⁶³. کاربران نابینا به کپچاهای صوتی وابسته‌اند، کاربران ناشنوا به کپچاهای تصویری. در طراحی کپچا باید **گزینه‌های جایگزین** امن نیز لحاظ شود تا مجبور نشویم برای رعایت حال کاربران، کپچا را تضعیف کنیم. مثلاً اگر یک کپچای تصویری پیچیده دارید، حتماً یک راهکار متنی یا صوتی مناسب هم برای افرادی که نمی‌توانند تصویر ببینند ارائه کنید (و آن راهکار جایگزین نیز نباید ساده و قابل حدس باشد). همچنین کپچاهای خیلی پیچیده‌ی چندمرحله‌ای ممکن است کاربران عادی را خسته کرده و نرخ رها کردن فرم‌ها را بالا ببرد ⁶⁴ ⁶⁵. یک طراحی خوب تعادلی بین امنیت و تجربه کاربری برقرار می‌کند.
- **استفاده از تحلیل رفتاری و بیومتریک‌های نامحسوس:** مدرن‌ترین روش‌های تمایز انسان و بات به جای چالش صریح، **بیومتریک رفتاری** و تحلیل پشت‌صحنه را ترجیح می‌دهند ⁶⁰ ⁶⁶. برای مثال، **ریتم تایپ کردن، الگوی حرکت ماوس، شیوه اسکرول کردن صفحه، ویژگی‌های دستگاه کاربر** و ده‌ها سیگنال دیگر می‌توانند یک امضای رفتاری منحصر به فرد ایجاد کنند. یک کپچای ایده‌آل شاید اصلاً به چشم کاربر نیاید؛ بلکه در پشت‌پرده این سیگنال‌ها را جمع‌آوری کرده و فقط در صورت مشاهده موارد غیرطبیعی (مثلاً حرکت ناگهانی و خطکش‌مانند ماوس که نشان از اسکرپت دارد، یا فشار کلیدهای کیبورد با فاصله زمانی بسیار ثابت) کاربر را به چالش اضافی وادار کند ⁶⁷ ⁶⁶. گوگل reCAPTCHA v3 دقیقاً چنین رویکردی دارد: به اکثر کاربران هیچ سؤال یا تصویری نشان نمی‌دهد و فقط یک امتیاز ریسک نامرئی تخصیص می‌دهد ⁶⁰. چنین مکانیزم‌هایی هرچند شکست‌ناپذیر نیستند، اما به مراتب دشوارتر قابل دور زدن‌اند چرا که حمله‌کننده باید **همزمان چندین فاکتور** را جعل کند (نه تنها پاسخ یک معما را بدهد).

۲. پیاده‌سازی صحیح و ایمن کپچا در برنامه

- **اعتبارسنجی کپچا حتماً در سمت سرور انجام شود:** هرگز نباید به اعتبارسنجی کپچا در سمت کاربر یا توسط جاوااسکریپت اکتفا کرد. همیشه پاسخ کاربر (یا توکن کپچا) باید به سرور ارسال و در آنجا تأیید شود که صحیح است. این شامل چک کردن امضای کپچا یا فراخوانی API سرویس‌دهنده (مثل API تأیید reCAPTCHA) است. در پیاده‌سازی سرویس‌های شخص ثالث، دقت کنید که **پاسخ صحیح را به درستی تفسیر کنید**؛ همان‌طور که دیدیم نباید صرفاً به کد وضعیت OK ۲۰۰ بسنده کرد و باید بدنه‌ی پاسخ (مثلاً فیلد `success` در JSON گوگل) را بررسی نمود ⁶⁸.

- **هر پاسخ کپچا فقط برای یکبار استفاده معتبر باشد:** یکی از اصول امنیت کپچا، **یک‌بار مصرف بودن** آن است. اگر کاربری یک کپچا را حل کرد، پاسخ یا توکن آن کپچا نباید مجدداً قابل استفاده باشد. بنابراین توکن‌ها یا شناسه‌های کپچا (مانند `CaptchaUniqueId`) را پس از اولین مصرف بلافاصله باطل کنید ⁶⁹. همچنین کپچا باید **عمر محدودی** داشته باشد (چند دقیقه یا حتی کمتر) و پس از انقضای زمان، دیگر پذیرفته نشود ⁷⁰. این کار جلوی حملات بازپخش پاسخ‌های قدیمی را می‌گیرد.

- **بستن تمام راه‌های ورودی به کپچا:** هنگام افزودن کپچا به یک عمل (مثلاً ثبت‌نام یا ورود)، مطمئن شوید در **همه‌ی مسیرهای ممکن** آن عمل بررسی صورت می‌گیرد. اگر فرم اصلی دارای کپچا است، یک API پنهان یا صفحه‌ی متفاوت نباید بدون کپچا پذیرای همان ورودی‌ها باشد. به عنوان مثال، اگر کپچا را در درخواست‌های `POST` بررسی می‌کنید، حتماً درخواست‌های معادل `GET` یا `JSON` را هم پوشش دهید تا مهاجم نتواند از مسیر دیگری وارد شود ⁷ ⁸. هرپارامتری که دخیل است (مانند نام کاربری یا ایمیل) را نیز در سرور اعتبارسنجی کنید تا تغییر آن موجب دور زدن کپچا نشود ¹⁰. علاوه بر این، **فیلدهای مخفی یا Honeypot** را هم در نظر بگیرید؛ اگر رباتی این فیلدهای مخفی را پر کرده بود، می‌توانید درخواستش را مردود کرده یا او را بلاک کنید. در کل، باید فرض کنید مهاجم همه تقلب‌های منطقی ممکن را امتحان خواهد کرد؛ پس از قبل همه‌ی درها را ببندید.

- **اعتبارسنجی پاسخی صحیح و قطعی داشته باشد:** در کد سمت سرور، منطق بررسی کپچا را **ساده و سرراست** بنویسید تا اشتباهات منطقی پیش نیاید. دیده شد که یک خطای کوچک در `if/else` می‌تواند کپچا را بی‌اثر کند ¹³. بنابراین حتماً کد خود را بازبینی و تست کنید. اگر از نمونه‌کدهای کپچا (مثل اسکریپت‌های آماده برای reCAPTCHA) استفاده می‌کنید، آن‌ها را مطابق مستندات رسمی پیاده کنید ⁷¹. به‌عنوان گام اضافی، می‌توانید **موارد عدم تطابق** را گزارش کنید (مثلاً اگر کاربری پاسخی داد که کپچا آن را غلط اعلام کرد، در لاگ ثبت شود). این نظارت کمک می‌کند متوجه شوید آیا ربات‌ها دارند چیزی را دور می‌زنند یا خیر.

- **بستن کلاینت بر اساس جلسه/IP در صورت لزوم:** یک راه محافظتی دیگر این است که **پاسخ کپچا را به جلسه کاربر یا IP او گره بزنید** ⁷². یعنی زمانی که یک کپچا تولید می‌کنید، آن را برای یک شناسه‌ی نشست و احتمالاً IP کاربر معتبر کنید. آنگاه اگر همان پاسخ کپچا از جلسه دیگری رسید، رد شود. این کار حملات بازپخش توسط مهاجمان دیگر را دشوار می‌کند (حتی اگر پاسخ کپچا را از جایی شنود کرده باشند). البته برخی سرویس‌های کپچا این کار را ذاتاً انجام می‌دهند (مثلاً توکن reCAPTCHA دربرگیرنده اطلاعات نشست است)، اما در کپچاهای ساده‌تر دستی باید خود پیاده‌سازی کنید.

- **مخفی نگه داشتن کلیدها و اسرار کپچا:** اگر از سرویس‌های کپچا مانند گوگل استفاده می‌کنید، حتماً **کلید خصوصی/سری** که برای اعتبارسنجی در سمت سرور استفاده می‌شود را ایمن نگه دارید ⁷³. این کلید نباید در کد سمت کاربر یا در مخازن عمومی منتشر شود. لو رفتن کلید خصوصی به مهاجم امکان می‌دهد مستقیماً به API سرویس کپچا درخواست اعتبارسنجی بزند و شاید راهی برای دور زدن یا سوءاستفاده پیدا کند. همچنین برای سرویس‌هایی که محدودیت استفاده با کلید دارند، افشای کلید موجب سوءاستفاده و سهمیه‌بندی بیش از حد خواهد شد.

۳. تدابیر مکمل برای تقویت امنیت کپچا

• **ترکیب کپچا با مکانیزم‌های دیگر امنیتی:** همان‌طور که کارشناسان اشاره می‌کنند، کپچا هرگز نباید تنها خط دفاعی شما باشد ⁷⁴. حتماً از سایر روش‌ها مانند محدودیت نرخ درخواست (Rate Limiting)، فیلتر کردن IP‌های مخرب یا کشورهای غیرمرتبط، استفاده از فایروال‌های برنامه‌های وب (WAF) و اعتبارسنجی‌های منطقه‌ای سمت سرور استفاده کنید ⁷⁵ ⁷⁶. به عنوان مثال، اگر کاربری ۱۰ بار پی‌پی‌کپچا را غلط حل کرد، می‌توان آن IP را موقتاً بلاک کرد یا سختی کپچا را افزایش داد. یا اگر از یک کشور مشخص ده‌ها هزار درخواست مشکوک می‌آید، کپچا به تنهایی کافی نیست و باید آن ترافیک را محدود یا مسدود کرد. **لایه‌بندی دفاعی** بدین معناست که حتی اگر یک لایه (مثل کپچا) شکست، لایه‌های دیگر جلوی نفوذ را بگیرند ⁷⁴.

• **نظارت بر الگوهای حل کپچا و استفاده از یادگیری ماشین برای تشخیص تقلب:** اگر وب‌سایت بزرگی دارید، ممکن است بخواهید آماری گردآوری کنید از اینکه کاربران چگونه کپچا را حل می‌کنند. برای مثال، میانگین زمانی که طول می‌کشد یک کاربر کپچا را حل کند چقدر است؟ نسبت موفقیت در اولین تلاش به چند درصد است؟ از چه کشورهایی یا چه دستگاه‌هایی درخواست‌ها می‌آید؟ این اطلاعات می‌تواند به ساختن **مدل‌های تشخیص بات** کمک کند. به عنوان نمونه، اگر سرویسی مشاهده کند که از یک IP خاص در عرض یک دقیقه ۵۰ کپچا با نرخ موفقیت ۹۸٪ حل شده، می‌تواند حدس بزند که این **انسان عادی نیست** (احتمالاً سرویس حل کپچاست) و آن IP یا رفتار را زیر نظر بگیرد یا محدود کند ⁷⁷ ⁷⁸. برخی سیستم‌ها می‌توانند **اثر انگشت‌های مشکوک** را به اشتراک بگذارند (مثلاً شبکه Cloudflare دارای پایگاه داده‌ای از مشتریان/مرورگرهای مورد اعتماد و مشکوک است). استفاده از چنین سرویس‌های امنیت ابری نیز می‌تواند کمک کند که مهاجمانی که از قبل شناسایی شده‌اند به راحتی نتوانند کپچاهای شما را دور بزنند.

• **به‌روزرسانی مداوم مکانیزم کپچا:** دنیای تقابل بات و کپچا دائماً در حال تحول است ⁷⁹. سازندگان کپچا راه‌های جدیدی برای سخت‌تر کردن چالش‌ها پیدا می‌کنند و مهاجمان راه‌های جدیدی برای شکست آن‌ها. بنابراین، فرض نکنید کپچایی که امروز امن است تا همیشه امن خواهد ماند. همواره به‌روز باشید؛ اخبار امنیتی و پژوهش‌های دانشگاهی درباره شکست کپچاها را دنبال کنید. اگر یک نوع کپچا دیگر کارایی ندارد (مثلاً کپچاهای متنی نسل قدیم تقریباً بی‌اثر شده‌اند)، به نوع جدیدتری مهاجرت کنید. نمونه این تکامل را می‌توان در حرکت از **کپچاهای متنی پیچیده** به **reCAPTCHA** دید که ابتدا بجای متن از عکس بهره گرفت و سپس در v3 حتی عکس هم حذف شد و به تحلیل رفتاری روی آورد ⁶⁶. اخیراً **Cloudflare** سرویس جایگزین کپچایی به نام **Turnstile** ارائه کرده که مدعی تجربه کاربری بهتر و امنیت بالاتر است ⁸⁰. هرچند گزارش شده که Turnstile نیز توسط بات‌ها تا حدی قابل دور زدن است ⁸¹، ولی مهم این است که همگام با فناوری‌های جدید حرکت کنیم. در کل، **کپچا باید پویا و سازگار با تهدیدات روز باشد** تا کارایی خود را حفظ کند.

طراحی یک کپچا کاملاً امن و غیرقابل دور زدن

آرمان نهایی یک طراح امنیت، ساختن کپچایی است که **هیچ راه نفوذی باقی نگذارد** و تحت هیچ شرایطی توسط ربات یا مهاجم دور زده نشود. اما آیا چنین چیزی امکان‌پذیر است؟ با توجه به مباحث گذشته، باید با واقع‌بینی اذعان کنیم که **هیچ کپچایی ۱۰۰٪ امن و شکست‌ناپذیر نیست** ⁷⁴. هر آزمونی که برای انسان قابل حل باشد، در نهایت با صرف زمان، هزینه یا هوش مصنوعی، توسط مهاجم نیز قابل حل یا دور زدن است ²⁷ ⁷⁸. با این وجود، می‌توان **ویژگی‌ها و اصولی** برشمرد که با رعایت همه‌ی آن‌ها، کپچا به بالاترین سطح امنیتی خود می‌رسد و شکست دادنش به شدت پرهزینه و سخت می‌شود. در ادامه این اصول را مرور می‌کنیم:

• **ترکیب چندین مؤلفه‌ی چالشی:** کپچایی که **چندوجهی** باشد، ربات‌ها را مجبور می‌کند در حوزه‌های مختلف مهارت کسب کنند. برای نمونه، می‌توان کپچایی طراحی کرد که کاربر را ملزم به انجام **یک فعالیت ذهنی و یک فعالیت بصری** کند. فرض کنید کپچا از کاربر می‌خواهد: «حاصل جمع ۵ و ۱۲ را حساب کنید و عدد را با حروف درون کادری که عکس یک گربه را نشان می‌دهد تایپ کنید.» در اینجا کاربر باید هم یک محاسبه انجام دهد (۱۵) و

هم تصویر یک گربه را از میان چند تصویر حیوانات تشخیص دهد و جواب را در کادر مربوطه وارد کند. حل چنین کپچایی برای انسان معمولی آسان است، ولی برای رباتی که می‌خواهد آن را دور بزند، نیاز به همزمان OCR (برای خواندن سوال)، درک زبان طبیعی (برای تفسیر سوال ریاضی)، بینایی ماشین (برای تشخیص تصویر گربه) و در نهایت تولید متن پاسخ دارد - مجموعه‌ای از چالش‌ها که هم‌زمان کردن آن‌ها بسیار دشوار است. البته این یک مثال ساده ترکیبی بود؛ طرح‌های پیچیده‌تری هم ممکن است ایجاد شود، مثل کپچایی که نیازمند **تعامل تعدی کاربر** با صفحه است (کشیدن شیء به سمت هدف مشخص، حل پازل‌های چندمرحله‌ای و جز آن). هرچه کپچا از کاربر کار متفاوت‌تر و بیشتری بخواهد، احتمال اسکرپیت‌نویسی موفق برای آن کاهش می‌یابد.

• **استفاده از آزمون‌های مبتنی بر هوش انسانی در برابر هوش مصنوعی:** ایده‌ای مطرح در طراحی کپچاهای نوین، طرح سوالاتی است که **هنوز برای هوش مصنوعی دشوار است اما برای انسان‌ها بدیهی یا ساده است**. برای مثال، درک مفهوم و زمینه یک تصویر ممکن است برای AI چالش‌برانگیز باشد. کپچایی را در نظر بگیرید که جمله‌ای توصیفی از یک سناریو می‌دهد و می‌پرسد کدامیک از دو تصویر این سناریو را نشان می‌دهد. یا کپچایی که از کاربر می‌خواهد احساسی را که یک چهره‌ی کارتونی نمایش می‌دهد تشخیص دهد (خشمگین، ناراحت، خوشحال و ...). این‌ها نوعی **آزمون تورینگ مصور** هستند که هنوز هوش مصنوعی عمومی در آن‌ها به دقت انسان نرسیده است. برخی پژوهش‌ها به سمت کپچاهای **منطق انسانی** رفته‌اند؛ مثلاً کاربر باید جوابی بدهد که مستلزم (Common Sense)常識 است که ماشین از آن بی‌بهره است. با اینکه مدل‌های زبانی بزرگ اخیر بخشی از این شکاف را پر کرده‌اند، اما ترکیب چالش‌های منطقی، بصری و شنیداری می‌تواند همچنان نقطه ضعف AI باشد. لذا برای طراحی کپچای ایده‌آل، باید روی مسائلی تمرکز کرد که **“سخت برای ماشین - آسان برای انسان”** باقی مانده‌اند.

• **بهره‌گیری از نظارت و یادگیری پویا:** یک کپچای امن می‌تواند خود یک **سیستم یادگیرنده** باشد. یعنی از تلاش‌های ناموفق یا الگوهای حمله درس بگیرد و مرتباً خود را تقویت کند. برای مثال، اگر تشخیص داد که باتی به‌طور مکرر در حال حدس زدن پاسخ‌هاست، دشواری کپچا را برای آن کلاینت خاص افزایش دهد (نمایش چالش‌های بیشتر یا پیچیده‌تر). یا اگر الگویی مشاهده کرد که نشان می‌دهد یک مدل OCR در حال شکست دادن آن است (مثلاً بسیاری پاسخ‌ها با زمان بسیار کم و دقت بالا از یک منبع می‌آید)، با افزودن پیچیدگی جدید (نوع نوین تازه در تصاویر یا تغییر فونت‌ها) مدل‌های مهاجم را از نو به چالش بکشد. **کپچای تطبیقی** که بتواند براساس تهدیدهای جاری تنظیم شود، طول عمر امنیتی بیشتری خواهد داشت. این نیازمند آن است که طراح کپچا فارغ از ارائه اولیه، دائماً سیستم را **مانیتور و به‌روزرسانی** کند. شرکت‌های بزرگ امنیتی دقیقاً همین کار را می‌کنند؛ برای نمونه، سرویس کپچای Arkose Labs یک پلتفرم چالش-پاسخ پویا ارائه داده که بر اساس سطح ریسک کاربر، از بازی‌های تعاملی گرفته تا پرسش‌های مختلف را طرح می‌کند و مرتباً این چالش‌ها را تغییر می‌دهد تا توسط بات‌ها شناسایی نشوند.

• **سخت‌تر کردن استفاده از حل انسانی برای مهاجم:** همانطور که گفتیم، پاشنه آشیل هر کپچایی امکان سپردن آن به انسان است. شاید نتوان جلوی این کاملاً را گرفت، اما می‌توان **هزینه و زمان حل انسانی** را بالا برد تا برای مهاجم غیربهره‌صرفه شود. برای مثال، اگر کپچایی طراحی کنیم که حل آن به جای ۱۰ ثانیه، ۲ دقیقه زمان ببرد، سرویس‌های حل کپچا برای مهاجم کند خواهند شد و صف انتظار طولانی ایجاد می‌کنند. برخی کپچاها برای این منظور از **معمای چندمرحله‌ای** بهره می‌برند (مثل سه دور سؤال پیاپی). یا کپچایی که نیازمند تعامل ممتد است (مثلاً کاربر باید در یک تصویر به دنبال اشیای متعدد بگردد و هر بار تأیید کند تا مرحله بعدی ظاهر شود). همچنین می‌توان **محدودیت زمانی پاسخ** گذاشت؛ مثلاً کپچا باید ظرف ۱۵ ثانیه حل شود وگرنه منقضی می‌شود. این کار استفاده از شبکه‌های کارگران انسانی را دشوارتر می‌کند، چون هماهنگ کردن آن‌ها در بازه‌های بسیار کوتاه سخت است. البته این تمهیدات روی کاربران واقعی نیز فشار می‌آورد، پس باید با احتیاط و سنجیده اعمال شوند تا باعث نارضایتی گسترده نشوند.

• **تلفیق کپچا با تأیید هویت در موارد حساس:** در بخش قبل اشاره شد که در امنیت چندلایه، کپچا تنها یک جزء است. اگر واقعاً بخواهیم تضمین کنیم که **هیچ‌کس تحت هیچ شرایطی نتواند عبور کند**، شاید لازم باشد برای عملیات بسیار حساس، علاوه بر کپچا از کاربران **روش‌های احراز هویت قوی‌تر** بخواهیم. برای مثال، در

هنگام ایجاد حساب کاربری جدید، بعد از حل کپچا، یک کد تأیید به ایمیل یا تلفن کاربر ارسال شود. این کار دیگر در قلمرو کپچا نیست بلکه احراز هویت دو مرحله‌ای است، اما ترکیب آن با کپچا قطعاً سد محکمی ایجاد می‌کند. هرچند باز هم اگر مهاجم دسترسی به آن ایمیل یا شماره داشته باشد یا... می‌تواند دور بزند، ولی داریم از حیطه بات‌های خودکار خارج می‌شویم. نکته این است که تفکر **“امنیت مطلق”** شاید تنها با ترکیب روش‌ها حاصل شود. کپچایی که کاملاً امن باشد، احتمالاً آن‌قدر برای کاربر سخت خواهد بود که کاربرپذیری را نابود می‌کند. بنابراین می‌توان مرزی ترسیم کرد و گفت: امن‌ترین کپچای ممکن، کپچایی است که در یک سطح قابل قبول از سهولت برای کاربر، حداکثر سختی را برای بات ایجاد کند.

در پایان، باید پذیرفت که جنگ بین طراحان کپچا و سازندگان بات پایانی ندارد.⁷⁹ هرچه **هوش مصنوعی** توانا تر شود، کپچاهای فعلی راحت‌تر حل می‌شوند و نیاز به خلاقیت و نوآوری بیشتری در طراحی کپچا خواهد بود.²³ شاید در آینده مفاهیمی چون **بیومتریک‌های منحصر به فرد انسانی** (مانند الگوی EEG مغز یا واکنش‌های فیزیولوژیک) به میان بیایند تا انسان و ماشین را جدا کنند؛ اما تا آن زمان، پیروی از اصولی که ذکر شد بهترین شانس ما برای ساخت **کپچایی مقاوم در برابر دور زدن** است.

جمع‌بندی

کپچاها به عنوان خط مقدم دفاع در برابر بات‌ها نقش مهمی در امنیت وب ایفا می‌کنند، اما تجربه نشان داده است که **هیچ کپچایی شکست‌ناپذیر نیست**.⁷⁴ ما در این مستند انواع روش‌های دور زدن کپچا - از بهره‌برداری از باگ‌های منطقی گرفته تا استفاده هوشمندانه از هوش مصنوعی و نیروی انسانی - را بررسی کردیم. همچنین راهکارهای متقابل برای هر دسته را برشمردیم؛ از **پیاده‌سازی درست و بدون باگ کپچا** گرفته تا **بهره‌گیری از سرویس‌های پیشرفته و تحلیل رفتاری** به منظور سخت‌تر کردن کار بات‌ها. نکته کلیدی آن است که امنیت کپچا را **در چارچوب یک رویکرد چندلایه** ببینیم.⁸² کپچا باید به خوبی طراحی و اجرا شود، اما علاوه بر آن باید با سایر مکانیزم‌های امنیتی همراه گردد و به طور مستمر در برابر تهدیدات جدید به‌روز شود.⁶⁶

در پاسخ به پرسش سوم - یعنی امکان طراحی کپچایی که به هیچ‌وجه قابل باایس نباشد - باید گفت این هدفی ایده‌آل و دشوار است. با این حال، با به‌کارگیری ترکیبی از **روش‌های ابتکاری** (چالش‌های ترکیبی، استفاده از آزمون‌های AI-hard، تطبیق پویا با حملات) می‌توان کپچایی ساخت که **دور زدن آن تا حد زیادی غیرممکن یا غیرمقرون به صرفه شود**. به تعبیر دیگر، هدف ما کاهش صرفه مهاجم است: یا ربات نتواند کپچا را حل کند، یا حل کردن آن چنان زمان‌بر و هزینه‌زا باشد که ارزش حمله را از بین ببرد.

در نهایت، **امنیت یک فرآیند مداوم است نه یک محصول نهایی**.⁸³ توسعه‌دهندگان باید همواره از آخرین تهدیدها و راه‌های دور زدن آگاه باشند و سیستم کپچای خود را تقویت کنند. با رعایت اصول ذکر شده در این مستند و بهره‌گیری از دانش روز، می‌توان اطمینان یافت که کپچاهای مورد استفاده، حداکثر سطح دفاعی ممکن را ارائه می‌کنند و به آسانی در برابر مهاجمان تسلیم نخواهند شد.

CAPTCHA Bypass Vulnerability - Insufficient 73 71 68 61 59 39 18 17 16 15 14 13 12 11 2 1

Attack Protection

[/https://blog.securelayer7.net/owasp-top-10-insufficient-attack-protection-7-captcha-bypass](https://blog.securelayer7.net/owasp-top-10-insufficient-attack-protection-7-captcha-bypass)

What is CAPTCHA? Types 82 79 78 77 76 75 74 67 66 65 64 63 62 60 44 41 29 28 27 24 23 3

and Examples

<https://www.wallarm.com/what/what-is-captcha-types-and-examples>

Methods to Bypass Captchas: A Deep Dive into Common Techniques | by 34 10 9 8 7 6 5 4

Dasmanish | Medium

<https://medium.com/@dasmanish6176/methods-to-bypass-captchas-a-deep-dive-into-common-techniques-309006f28923>

Unmasking a Captcha Bypass Vulnerability: Step-by-Step Walkthrough | by 83 72 70 69 22 21 20 19

Vishal Sharma | Medium

[-https://medium.com/@vishalsharma445500/unmasking-a-captcha-bypass-vulnerability-step-by-step-walkthrough-6131519a3788](https://medium.com/@vishalsharma445500/unmasking-a-captcha-bypass-vulnerability-step-by-step-walkthrough-6131519a3788)

How To Solve CAPTCHAs with Python | ScrapeOps 46 45 43 42 31 30 26 25

[/https://scrapeops.io/python-web-scraping-playbook/python-how-to-solve-captchas](https://scrapeops.io/python-web-scraping-playbook/python-how-to-solve-captchas)

Bypassing ReCaptcha in 2024? : r/webscraping - Reddit 32

[/https://www.reddit.com/r/webscraping/comments/1af1u0h/bypassing_recaptcha_in_2024](https://www.reddit.com/r/webscraping/comments/1af1u0h/bypassing_recaptcha_in_2024)

How to Bypass CAPTCHA in Scraping: 10 Methods for 2025 58 57 56 55 51 50 49 38 37 33

[/https://marsproxies.com/blog/bypass-captcha](https://marsproxies.com/blog/bypass-captcha)

Proven Ways to Bypass CAPTCHA in Python 5 54 53 52 48 47 36 35

<https://scrapfly.io/blog/posts/how-to-bypass-captcha-while-web-scraping>

... ChatGPT Tricked Into Solving CAPTCHAs: Security Risks for AI and 40

[/https://www.esecurityplanet.com/news/chatgpt-tricked-into-solving-captchas-security-risks-for-ai-and-enterprise-systems](https://www.esecurityplanet.com/news/chatgpt-tricked-into-solving-captchas-security-risks-for-ai-and-enterprise-systems)

Cloudflare Turnstile | CAPTCHA Replacement Solution 80

[/https://www.cloudflare.com/application-services/products/turnstile](https://www.cloudflare.com/application-services/products/turnstile)

Bypass Cloudflare with Puppeteer (2025 Guide) - Browserless 81

<https://www.browserless.io/blog/bypass-cloudflare-with-puppeteer>